

PYTHON · TKINTER · PROGRAMACIÓN ESTRUCTURADA

Sustracción de Matrices

Tres métodos explicados línea por línea, con diagramas de flujo, interfaz gráfica en Tkinter y menú de tamaño 2×2, 3×3 y N×N

The diagram illustrates the subtraction of matrix B from matrix A. Matrix A is a 2x2 matrix with values 5, 3, 2, and 8. Matrix B is a 2x2 matrix with values 1, 1, 4, and 2. The result, matrix C, is a 2x2 matrix with values 4, 2, -2, and 6. Matrix C is highlighted with an orange border.

A	
5	3
2	8

—

B	
1	1
4	2

=

C	
4	2
-2	6

Guía técnica · incluye código fuente completo (backend + frontend)

Índice

1. Introducción — ¿qué es la resta de matrices?
2. Arquitectura general de la aplicación
3. Método 1 — Bucles anidados Backend + Frontend
4. Método 2 — Comprensión de listas Backend + Frontend
5. Método 3 — NumPy (vectorizado) Backend + Frontend
6. Aplicación integrada final — menú de tamaño y de método
7. Tabla comparativa de los 3 métodos
8. Glosario de términos de Python usados
9. Conclusiones

1. Introducción — ¿Qué es la resta de matrices?

Una **matriz** es una tabla de números organizada en filas y columnas. La **resta de matrices** ($A - B$) solo se puede hacer si ambas matrices tienen **exactamente el mismo tamaño** (mismo número de filas y de columnas). El resultado es una nueva matriz C donde cada casilla se calcula restando, uno a uno, los elementos que están en la misma posición:

$$C[i][j] = A[i][j] - B[i][j] \quad \text{para toda fila } i \text{ y columna } j$$

Es decir: la casilla de arriba-izquierda de A menos la casilla de arriba-izquierda de B da la casilla de arriba-izquierda de C , y así sucesivamente con cada posición. No se "mezclan" filas con columnas distintas: cada resta es siempre entre elementos que ocupan el mismo lugar.

En este documento se construye, con **Python** puro y la librería gráfica **Tkinter**, una aplicación que resuelve este problema de **tres formas distintas** (tres algoritmos diferentes que llegan al mismo resultado), cada una con su propia lógica interna:

- **Método 1 — Bucles anidados**: la forma más explícita, recorriendo la matriz casilla por casilla con dos `for`.
- **Método 2 — Comprensión de listas**: el mismo recorrido, pero escrito en una sola expresión compacta (muy usado en Python "idiomático").
- **Método 3 — NumPy (vectorizado)**: se delega el cálculo a la librería NumPy, que resuelve la resta internamente sin que nosotros escribamos ningún bucle.

Cada método se presenta dividido en dos partes, tal como se explican por separado en las siguientes secciones:

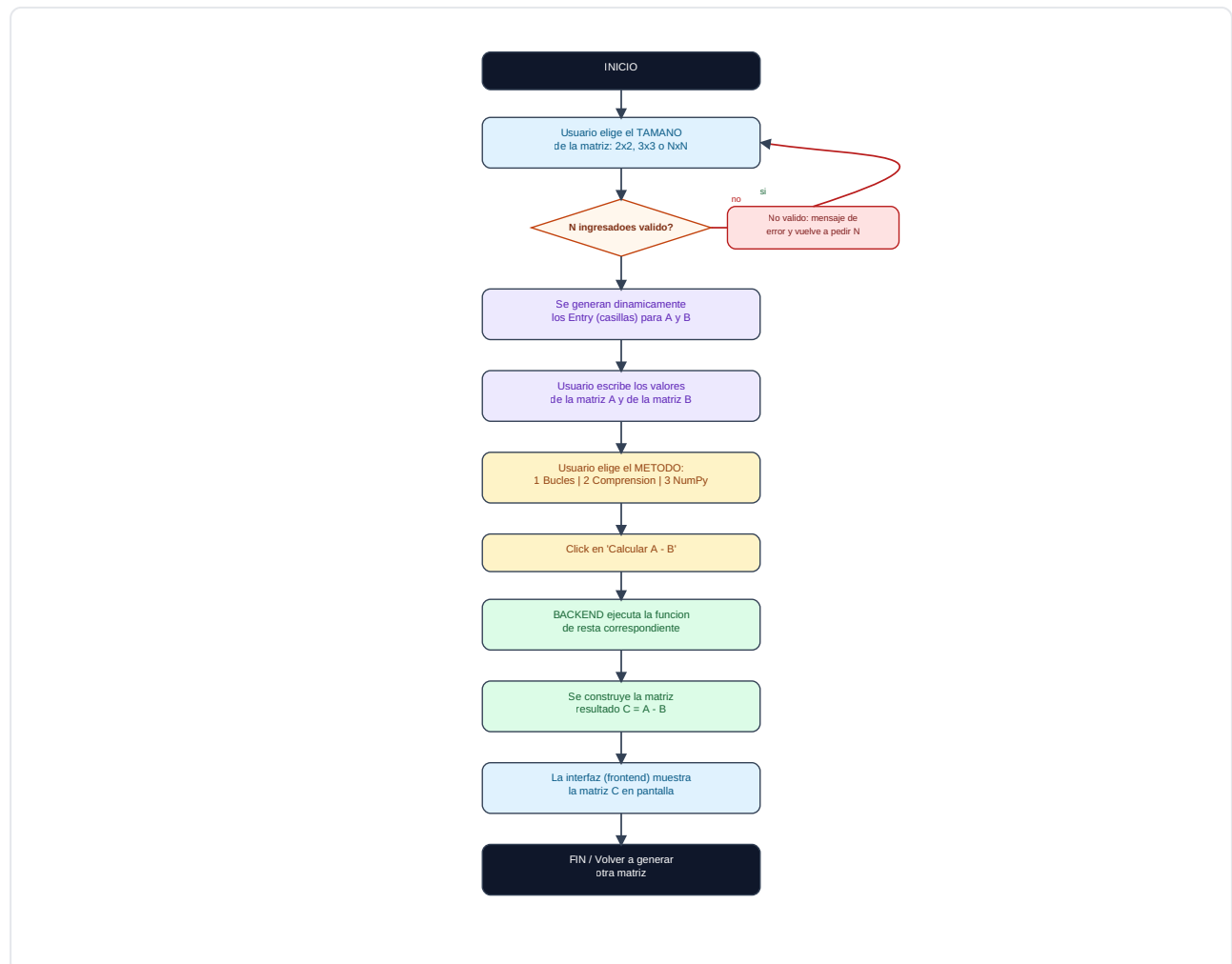
- **Backend**: la función pura de Python que recibe dos matrices y devuelve la resta. No dibuja nada en pantalla, solo calcula.
- **Frontend**: la ventana de Tkinter (botones, casillas de texto, menús) que permite al usuario escribir las matrices, elegir el tamaño (2×2 , 3×3 o $N \times N$) y ver el resultado.

Ejemplo numérico rápido (2x2):

$A = \begin{bmatrix} 5 & 3 \\ 2 & 8 \end{bmatrix}$ $B = \begin{bmatrix} 1 & 1 \\ 4 & 2 \end{bmatrix}$ $A - B = \begin{bmatrix} 5-1 & 3-1 \\ 2-4 & 8-2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ -2 & 6 \end{bmatrix}$

2. Arquitectura general de la aplicación

Antes de entrar a cada método, este diagrama muestra **el recorrido completo** que sigue el programa desde que se abre la ventana hasta que se muestra el resultado en pantalla. Las flechas indican el orden en que ocurren las cosas; el rombo naranja representa una decisión (una validación) que puede desviar el flujo hacia un mensaje de error.



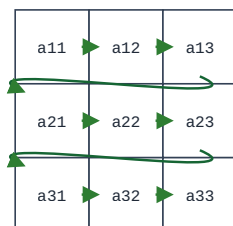
Este flujo es el mismo para los tres métodos: lo único que cambia es **qué función de backend** se ejecuta en el paso "BACKEND ejecuta la función de resta correspondiente". Por eso, en la aplicación integrada final (sección 6), ese paso se resuelve con un simple menú de selección.

3. Método 1 — Bucles anidados (for tradicional)

3.1 Idea del método

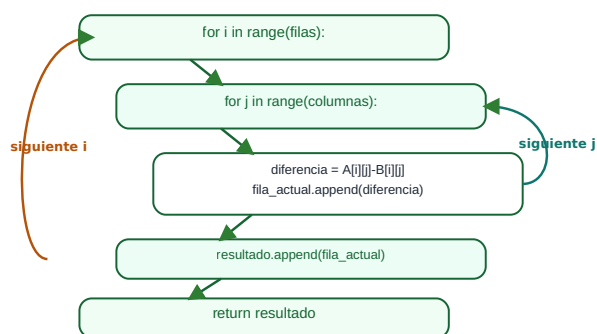
Es la forma más "manual" y explícita de resolver el problema: se usan **dos bucles for uno dentro del otro**. El bucle externo (*i*) recorre las filas; por cada fila, el bucle interno (*j*) recorre las columnas. En cada combinación (*i*, *j*) se calcula una resta y se guarda. Es exactamente como resolver la resta "a mano", casilla por casilla, de izquierda a derecha y de arriba hacia abajo.

Orden en que 'j' e 'i' recorren A



Flecha verde = avanza j (misma fila). Flecha curva = termina la fila, i avanza y j vuelve a 0.

Diagrama de flujo del código (equivalente)



El diagrama de la izquierda muestra el **orden real** en que se visita cada casilla (flechas verdes = avanza la columna *j*; flechas curvas = termina una fila y empieza la siguiente, avanza *i*). A la derecha, el mismo recorrido pero representado como **diagrama de flujo del código**: se ve claramente cómo el flujo "entra" al bucle de *j*, vuelve a él varias veces ("siguiente j"), y solo cuando termina esa fila regresa al bucle de *i* ("siguiente i") para pasar a la fila de abajo.

3.2 Backend — código completo

Esta función NO usa Tkinter ni dibuja nada: solo recibe dos matrices y devuelve la resta. Se puede probar sola, sin abrir ninguna ventana.

```
def restar_matrices_bucles(A, B):
    """
    Resta la matriz B a la matriz A usando bucles for anidados (el metodo
    mas explicito: se visita casilla por casilla, como si se recorriera
    la matriz con el dedo, fila por fila).

    Parametros:
        A -> lista de listas (matriz minuendo)
        B -> lista de listas (matriz sustraendo). Debe tener el mismo
            numero de filas y columnas que A.

    Retorna:
        resultado -> lista de listas con la matriz diferencia (A - B)
    """
    filas = len(A)          # numero de filas de A (cuantas listas internas tiene A)
    columnas = len(A[0])    # numero de columnas (cuantos elementos tiene la primera fila)
    resultado = []          # aqui se iran guardando las filas ya calculadas

    for i in range(filas):  # i recorre cada fila: 0, 1, 2, ...
        fila_actual = []    # fila nueva y vacia para la fila i del resultado
        for j in range(columnas):
            diferencia = A[i][j] - B[i][j]
            fila_actual.append(diferencia)
            # resta el elemento (i,j) de A menos el (i,j) de B
            # agrega ese numero al final de la fila_actual
            # cuando la fila esta completa, se agrega al resultado
        resultado.append(fila_actual)

    return resultado        # se entrega la matriz diferencia completa
```

Explicación línea por línea

Línea	Código	Qué hace / qué significa
1	def restar_matrices_bucles(A, B):	Define la función. Recibe dos parámetros: A y B, que son las dos matrices (listas de listas) a restar.
9	filas = len(A)	len(A) cuenta cuántos elementos tiene la lista A. Como A es una lista de filas, esto da el número de filas .
10	columnas = len(A[0])	A[0] es la primera fila de A (una lista). Su longitud es el número de columnas .
11	resultado = []	Crea una lista vacía donde se irán guardando, una por una, las filas ya calculadas de la matriz resultado.
13	for i in range(filas):	range(filas) genera los números 0, 1, 2, ... hasta filas-1. La variable i representa el índice de la fila actual .
14	fila_actual = []	Por cada fila nueva que se va a construir, se crea una lista vacía que irá guardando los valores de esa fila.
15	for j in range(columnas):	Bucle interno: j recorre cada columna dentro de la fila i actual, de 0 a columnas-1.
16	diferencia = A[i][j] - B[i][j]	Toma el elemento en la posición (i, j) de A y le resta el elemento en la misma posición (i, j) de B. A[i][j] significa "fila i, columna j de A".
17	fila_actual.append(diferencia)	.append() agrega el número calculado al final de fila_actual. Así la fila se va llenando columna por columna.
18	resultado.append(fila_actual)	Cuando el bucle interno (j) termina, la fila ya está completa y se agrega como un nuevo elemento a la lista resultado.

Línea	Código	Qué hace / qué significa
20	<code>return resultado</code>	Una vez que el bucle externo (i) también termina, se entrega la matriz resultado completa a quien llamó la función.

3.3 Frontend — código completo (Tkinter)

Esta parte construye la ventana: el menú para elegir el tamaño (2×2 / 3×3 / $N \times N$), las casillas donde el usuario escribe los números de A y B, el botón "Calcular", y el área donde se muestra el resultado. Internamente llama siempre a `restar_matrices_bucles` (el backend de arriba).

```

import tkinter as tk
from tkinter import messagebox
from m1_backend import restar_matrices_bucles # trae la funcion que hace la resta real

class VentanaMetodo1:
    """Interfaz grafica (Tkinter) para el Metodo 1: bucles anidados."""

    def __init__(self, raiz):
        self.raiz = raiz
        self.raiz.title("Metodo 1 - Resta de Matrices con Bucles Anidados")

        self.n = tk.IntVar(value=2) # tamaño elegido en el menu (2, 3 o -1 para NxN)
        self.entradas_a = [] # aqui se guardaran los widgets Entry de la matriz A
        self.entradas_b = [] # aqui se guardaran los widgets Entry de la matriz B
        self.n_actual = 2 # tamaño realmente usado al generar los campos

        self.marco_menu = tk.Frame(raiz, padx=10, pady=10)
        self.marco_menu.pack()
        self.marco_matrices = tk.Frame(raiz, padx=10, pady=10)
        self.marco_matrices.pack()
        self.marco_resultado = tk.Frame(raiz, padx=10, pady=10)
        self.marco_resultado.pack()

        self._crear_menu_tamano()

# ----- MENU: elegir tamaño 2x2 / 3x3 / NxN -----
def _crear_menu_tamano(self):
    tk.Label(self.marco_menu, text="1) Elige el tamaño de la matriz:",
            font=("Arial", 11, "bold")).grid(row=0, column=0, columnspan=4, sticky="w")

    tk.Radiobutton(self.marco_menu, text="2 x 2", variable=self.n, value=2).grid(row=1, column=0)
    tk.Radiobutton(self.marco_menu, text="3 x 3", variable=self.n, value=3).grid(row=1, column=1)
    tk.Radiobutton(self.marco_menu, text="N x N ->", variable=self.n, value=-1).grid(row=1, column=2)

    self.entrada_n = tk.Entry(self.marco_menu, width=4) # campo para escribir N cuando es NxN
    self.entrada_n.grid(row=1, column=3)

    tk.Button(self.marco_menu, text="Generar matrices",
            command=self._dibujar_campos).grid(row=2, column=0, columnspan=4, pady=6)

# ----- Lee y valida el tamaño N elegido -----
def _obtener_n(self):
    if self.n.get() == -1: # si el usuario marco la opcion "NxN"
        valor = self.entrada_n.get().strip()
        if not valor.isdigit() or int(valor) < 1:
            messagebox.showerror("Error", "Ingresa un entero N valido (N >= 1).")
            return None
        return int(valor)
    return self.n.get() # si eligio 2 o 3, se devuelve directo

# ----- Dibuja los recuadros (Entry) para llenar A y B -----
def _dibujar_campos(self):
    for widget in self.marco_matrices.winfo_children(): # limpia lo que hubiera antes
        widget.destroy()

    n = self._obtener_n()
    if n is None:
        return
    self.n_actual = n
    self.entradas_a = []
    self.entradas_b = []

    tk.Label(self.marco_matrices, text="Matriz A", font=("Arial", 10, "bold"))
        .grid(row=0, column=0, columnspan=n)
    tk.Label(self.marco_matrices, text="Matriz B", font=("Arial", 10, "bold"))
        .grid(row=0, column=n + 1, columnspan=n)

    for i in range(n): # crea, fila por fila...
        fila_a, fila_b = [], []
        for j in range(n): # ...columna por columna, un Entry por casilla
            celda_a = tk.Entry(self.marco_matrices, width=5, justify="center")
            celda_a.grid(row=i + 1, column=j, padx=2, pady=2)
            celda_a.insert(0, "0")
            fila_a.append(celda_a)

            celda_b = tk.Entry(self.marco_matrices, width=5, justify="center")
            celda_b.grid(row=i + 1, column=n + 1 + j, padx=2, pady=2)

```

```

        celda_b.insert(0, "0")
        fila_b.append(celda_b)
        self.entradas_a.append(fila_a)
        self.entradas_b.append(fila_b)

tk.Button(self.marco_matrices, text="Calcular A - B", bg="#2e7d32", fg="white",
          command=self._calcular).grid(row=n + 2, column=0, columnspan=2 * n + 1, pady=10)

# ----- Toma los numeros escritos y llama al backend -----
def _calcular(self):
    n = self.n_actual
    try:
        A = [[float(self.entradas_a[i][j].get()) for j in range(n)] for i in range(n)]
        B = [[float(self.entradas_b[i][j].get()) for j in range(n)] for i in range(n)]
    except ValueError:
        messagebox.showerror("Error", "Todos los valores deben ser numeros.")
        return

    resultado = restar_matrices_bucles(A, B) # <-- aqui se usa el backend (Metodo 1)
    self._mostrar_resultado(resultado)

# ----- Muestra la matriz resultado en pantalla -----
def _mostrar_resultado(self, resultado):
    for widget in self.marco_resultado.winfo_children():
        widget.destroy()
    tk.Label(self.marco_resultado, text="Resultado A - B",
            font=("Arial", 10, "bold")).pack()
    texto = "\n".join(" ".join(f"{val:g}" for val in fila) for fila in resultado)
    tk.Label(self.marco_resultado, text=texto, font=("Courier", 12), justify="left").pack()

if __name__ == "__main__":
    raiz = tk.Tk()
    VentanaMetodo1(raiz)
    raiz.mainloop()

```

Explicación línea por línea (partes clave)

Línea	Código	Qué hace / qué significa
3	<code>from m1_backend import restar_matrices_bucles</code>	Importa la función del backend. Así el frontend nunca calcula nada por su cuenta: solo pide números y llama a esta función.
6	<code>class VentanaMetodo1:</code>	Define una clase : un molde para crear la ventana. Agrupa en un solo lugar los widgets (botones, casillas) y las funciones que reaccionan a los clics.
9	<code>self.n = tk.IntVar(value=2)</code>	<code>IntVar</code> es una "caja" especial de Tkinter que guarda un número entero y que los widgets (como los Radiobutton) pueden leer y modificar automáticamente.
10-11	<code>self.entradas_a = [] self.entradas_b = []</code>	Listas donde se guardarán las referencias a cada casilla de texto (<code>Entry</code>) de las matrices A y B, para poder leerlas después.
24-26	<code>tk.Radiobutton(..., value=2) tk.Radiobutton(..., value=3) tk.Radiobutton(..., value=-1)</code>	Crea los botones de opción del menú de tamaño: 2×2, 3×3, o "NxN" (con valor especial -1 que activa el campo de texto para escribir N).
35-40	<code>def _obtener_n(self):</code>	Función que decide qué número N usar: si el usuario marcó "NxN" lee el <code>Entry</code> de texto y valida que sea un entero positivo; si no, devuelve directamente 2 o 3.
44-46	<code>for widget in self.marco_matrices.winfo_children(): widget.destroy()</code>	<code>winfo_children()</code> devuelve todos los widgets que hay dentro del marco. El bucle los borra uno por uno para poder dibujar una matriz de un nuevo tamaño sin que se amontonen las anteriores.

Línea	Código	Qué hace / qué significa
58-70	<pre>for i in range(n): for j in range(n): celda_a = tk.Entry(...)</pre>	Dos bucles anidados generan dinámicamente los recuadros de texto: uno por cada casilla de A y uno por cada casilla de B, según el tamaño N elegido.
79-86	<pre>def _calcular(self):</pre>	Se ejecuta al presionar el botón "Calcular". Lee el texto de cada <code>Entry</code> , lo convierte a número con <code>float()</code> , arma las listas A y B, y llama al backend.
89	<pre> resultado = restar_matrices_bucles(A, B)</pre>	Aquí se conecta el frontend con el backend: se le pasan las matrices leídas de pantalla y se recibe la matriz diferencia ya calculada.
94-99	<pre>def _mostrar_resultado(self, resultado):</pre>	Convierte la lista de listas del resultado en texto legible y lo muestra en una <code>Label</code> (etiqueta) debajo del formulario.

Flujo de ejecución paso a paso:

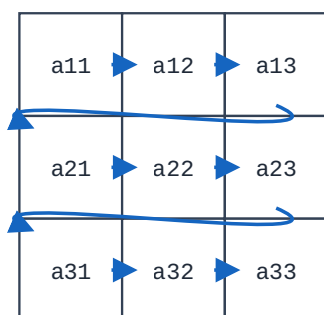
1. Se crea la ventana y se llama `VentanaMetodo1(raiz)` → se construye el menú de tamaño.
2. El usuario elige 2×2, 3×3 o escribe un N y presiona "Generar matrices" → se llama `_dibujar_campos()`.
3. `_dibujar_campos()` limpia la pantalla anterior y crea N×N casillas para A y N×N para B.
4. El usuario escribe los números y presiona "Calcular A - B" → se llama `_calcular()`.
5. `_calcular()` lee cada `Entry`, arma las listas A y B, y llama a `restar_matrices_bucles(A, B)`.
6. El backend recorre la matriz con los dos `for` (ver diagrama 3.1) y devuelve la matriz resultado.
7. `_mostrar_resultado()` convierte el resultado en texto y lo pinta en pantalla.

4. Método 2 — Comprensión de listas (*list comprehension*)

4.1 Idea del método

Este método **recorre exactamente las mismas casillas, en el mismo orden** que el Método 1 (fila por fila, columna por columna). La diferencia no está en la lógica matemática, sino en **cómo se escribe** el código: en vez de usar dos bucles `for` con `append()`, se usa una **comprensión de listas**, una construcción propia de Python que arma listas nuevas en una sola línea.

El recorrido es igual al Metodo 1...



```
resultado = [  
    [A[i][j]-B[i][j] for j in ...]  
    for i in ...  
]
```

...pero en vez de escribir cada paso por separado, Python arma la matriz completa en una única expresión: la comprensión «interna» construye cada fila y el bucle «externo» recorre las filas.

A la izquierda, el mismo recorrido fila por fila del Método 1. A la derecha, cómo se "empaqueta" ese mismo recorrido dentro de una sola expresión: la comprensión **interna** (`for j in ...`) arma cada fila, y la comprensión **externa** (`for i in ...`) repite ese armado por cada fila de la matriz.

4.2 Backend — código completo

BACKEND m2_backend.py — Método 2: Backend

```
def restar_matrices_comprension(A, B):
    """
    Resta la matriz B a la matriz A usando comprension de listas
    (list comprehension). Hace exactamente lo mismo que el Metodo 1
    (dos bucles anidados), pero escrito en una sola expresion compacta.

    Parametros:
        A -> lista de listas (matriz minuendo)
        B -> lista de listas (matriz sustraendo), mismo tamaño que A

    Retorna:
        resultado -> lista de listas con la matriz diferencia (A - B)
    """
    filas = len(A)          # cantidad de filas
    columnas = len(A[0])    # cantidad de columnas

    resultado = [
        [A[i][j] - B[i][j] for j in range(columnas)] # comprension INTERNA -> construye una fila
        for i in range(filas)                        # comprension EXTERNA -> recorre las filas
    ]
    # Se lee de derecha a izquierda / de afuera hacia adentro:
    # "for i in range(filas)" -> por cada fila i...
    # "for j in range(columnas)" -> ...por cada columna j de esa fila...
    # "A[i][j] - B[i][j]" -> ...calcula la resta de esa casilla.

    return resultado
```

Explicación línea por línea

Línea	Código	Qué hace / qué significa
1	def restar_matrices_comprension(A, B):	Misma firma que el Método 1: recibe A y B, devuelve la matriz resultado. Por fuera se comporta igual; por dentro cambia la forma de escribir el recorrido.
11-12	filas = len(A) columnas = len(A[0])	Igual que en el Método 1: se calculan las dimensiones antes de recorrer la matriz.
14	resultado = [	Se abre una lista por comprensión : en vez de usar <code>append()</code> dentro de un <code>for</code> tradicional, Python construye la lista completa en una sola expresión.
15	[A[i][j] - B[i][j] for j in range(columnas)]	Esta es la comprensión interna : construye <i>una fila completa</i> , recorriendo <code>j</code> de 0 a <code>columnas-1</code> y calculando la resta en cada posición. Es equivalente al bucle interno del Método 1.
16	for i in range(filas)	Esta es la comprensión externa : se repite la construcción de la fila (la línea de arriba) una vez por cada valor de <code>i</code> , es decir, una vez por fila. Equivale al bucle externo del Método 1.
17	]	Cierra la lista por comprensión. En este punto, <code>resultado</code> ya es la lista de listas completa, calculada en una sola instrucción.
24	return resultado	Se entrega el resultado, exactamente igual que en el Método 1.

Cómo leer una comprensión de listas (de afuera hacia adentro):

```
[ [A[i][j]-B[i][j] for j in range(columnas)] for i in range(filas) ]
```

1 Se lee primero el `for` de más a la derecha/afuera: "por cada fila i..."

- 2 Luego el siguiente `for` , más adentro: *"...por cada columna j de esa fila..."*
- 3 Y al final, lo que está más a la izquierda es lo que realmente se calcula: *"...calcula $A[i][j]-B[i][j]$ "*.

4.3 Frontend — código completo (Tkinter)

El frontend del Método 2 tiene **la misma interfaz gráfica** que el Método 1 (mismo menú de tamaño 2×2 / 3×3 / $N \times N$, mismas casillas), porque la parte visual no depende de cómo se calcula la resta por dentro. La única diferencia real está en una sola línea: qué función de backend se llama al presionar "Calcular".

```

import tkinter as tk
from tkinter import messagebox
from m2_backend import restar_matrices_comprension # backend del Metodo 2

class VentanaMetodo2:
    """Interfaz grafica (Tkinter) para el Metodo 2: comprension de listas."""

    def __init__(self, raiz):
        self.raiz = raiz
        self.raiz.title("Metodo 2 - Resta de Matrices con Comprension de Listas")

        self.n = tk.IntVar(value=2)
        self.entradas_a = []
        self.entradas_b = []
        self.n_actual = 2

        self.marco_menu = tk.Frame(raiz, padx=10, pady=10)
        self.marco_menu.pack()
        self.marco_matrices = tk.Frame(raiz, padx=10, pady=10)
        self.marco_matrices.pack()
        self.marco_resultado = tk.Frame(raiz, padx=10, pady=10)
        self.marco_resultado.pack()

        self._crear_menu_tamano()

    def _crear_menu_tamano(self):
        tk.Label(self.marco_menu, text="1) Elige el tamaño de la matriz:",
                 font=("Arial", 11, "bold")).grid(row=0, column=0, columnspan=4, sticky="w")
        tk.Radiobutton(self.marco_menu, text="2 x 2", variable=self.n, value=2).grid(row=1, column=0)
        tk.Radiobutton(self.marco_menu, text="3 x 3", variable=self.n, value=3).grid(row=1, column=1)
        tk.Radiobutton(self.marco_menu, text="N x N ->", variable=self.n, value=-1).grid(row=1, column=2)
        self.entrada_n = tk.Entry(self.marco_menu, width=4)
        self.entrada_n.grid(row=1, column=3)
        tk.Button(self.marco_menu, text="Generar matrices",
                  command=self._dibujar_campos).grid(row=2, column=0, columnspan=4, pady=6)

    def _obtener_n(self):
        if self.n.get() == -1:
            valor = self.entrada_n.get().strip()
            if not valor.isdigit() or int(valor) < 1:
                messagebox.showerror("Error", "Ingresa un entero N valido (N >= 1).")
                return None
            return int(valor)
        return self.n.get()

    def _dibujar_campos(self):
        for widget in self.marco_matrices.winfo_children():
            widget.destroy()
        n = self._obtener_n()
        if n is None:
            return
        self.n_actual = n
        self.entradas_a, self.entradas_b = [], []

        tk.Label(self.marco_matrices, text="Matriz A", font=("Arial", 10, "bold"))
            .grid(row=0, column=0, columnspan=n)
        tk.Label(self.marco_matrices, text="Matriz B", font=("Arial", 10, "bold"))
            .grid(row=0, column=n + 1, columnspan=n)

        for i in range(n):
            fila_a, fila_b = [], []
            for j in range(n):
                celda_a = tk.Entry(self.marco_matrices, width=5, justify="center")
                celda_a.grid(row=i + 1, column=j, padx=2, pady=2)
                celda_a.insert(0, "0")
                fila_a.append(celda_a)

                celda_b = tk.Entry(self.marco_matrices, width=5, justify="center")
                celda_b.grid(row=i + 1, column=n + 1 + j, padx=2, pady=2)
                celda_b.insert(0, "0")
                fila_b.append(celda_b)
            self.entradas_a.append(fila_a)
            self.entradas_b.append(fila_b)

        tk.Button(self.marco_matrices, text="Calcular A - B", bg="#1565c0", fg="white",
                  command=self._calcular).grid(row=n + 2, column=0, columnspan=2 * n + 1, pady=10)

```

```

def _calcular(self):
    n = self.n_actual
    try:
        A = [[float(self.entradas_a[i][j].get()) for j in range(n)] for i in range(n)]
        B = [[float(self.entradas_b[i][j].get()) for j in range(n)] for i in range(n)]
    except ValueError:
        messagebox.showerror("Error", "Todos los valores deben ser numeros.")
        return

    resultado = restar_matrices_comprension(A, B) # <-- backend del Metodo 2
    self._mostrar_resultado(resultado)

def _mostrar_resultado(self, resultado):
    for widget in self.marco_resultado.winfo_children():
        widget.destroy()
    tk.Label(self.marco_resultado, text="Resultado A - B",
             font=("Arial", 10, "bold")).pack()
    texto = "\n".join(" ".join(f"{val:g}" for val in fila) for fila in resultado)
    tk.Label(self.marco_resultado, text=texto, font=("Courier", 12), justify="left").pack()

if __name__ == "__main__":
    raiz = tk.Tk()
    VentanaMetodo2(raiz)
    raiz.mainloop()

```

Diferencia clave respecto al Frontend del Método 1

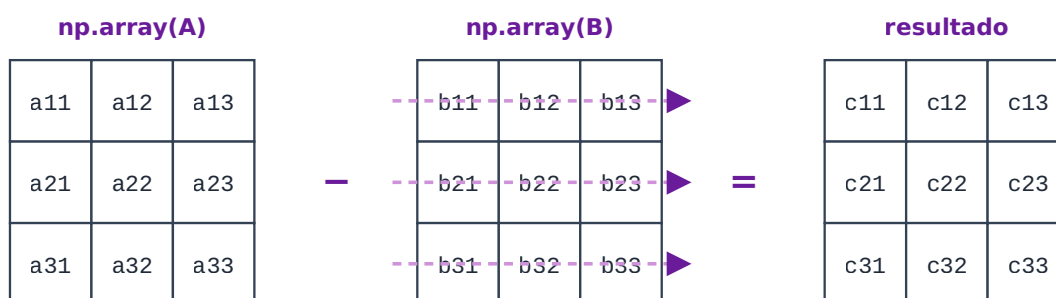
Línea	Código	Qué hace / qué significa
3	from m2_backend import restar_matrices_comprension	Cambia el import: ahora se trae la función que usa comprensión de listas en vez de bucles explícitos.
83	resultado = restar_matrices_comprension(A, B)	Esta es la única línea de lógica que cambia frente al Método 1: se llama a la función del backend del Método 2. Todo lo demás (leer las casillas, mostrar el resultado) es idéntico.

5. Método 3 — NumPy (vectorización)

5.1 Idea del método

NumPy es una librería externa de Python especializada en matemática con matrices. En vez de escribir nosotros mismos el recorrido casilla por casilla, se convierten las listas en **arrays de NumPy** y se usa directamente el operador de resta (`-`). NumPy resuelve toda la operación internamente, con código ya compilado (en C), procesando la matriz completa de forma **vectorizada** en vez de elemento por elemento desde el intérprete de Python.

Las 9 restas ocurren "a la vez": no hay un bucle for visible en el código Python.



NumPy vectoriza la operación: internamente usa código en C ya compilado que recorre toda la memoria de forma optimizada, sin que el intérprete de Python ejecute un bucle línea por línea como en los Métodos 1 y 2.

A diferencia de los diagramas de los Métodos 1 y 2 —donde las flechas mostraban un **orden** de recorrido (primero esta casilla, luego la siguiente)—, aquí todas las flechas se dibujan "a la vez": conceptualmente, NumPy calcula las 9 restas de forma simultánea/optimizada, sin que el código Python que nosotros escribimos controle ese orden.

5.2 Backend — código completo

BACKEND m3_backend.py — Método 3: Backend (NumPy)

```
import numpy as np # libreria externa especializada en calculo numerico y vectores/matrices

def restar_matrices_numpy(A, B):
    """
    Resta la matriz B a la matriz A usando NumPy (vectorizacion).
    A diferencia del Metodo 1 y el Metodo 2, aqui NO escribimos ningun
    bucle: NumPy resuelve internamente (en codigo C, muy optimizado)
    la resta de todas las casillas "al mismo tiempo".

    Parametros:
        A -> lista de listas (o array) - matriz minuendo
        B -> lista de listas (o array) - matriz sustraendo

    Retorna:
        resultado -> lista de listas con la matriz diferencia (A - B)
    """
    matriz_a = np.array(A) # convierte la lista de listas A en un array de NumPy
    matriz_b = np.array(B) # convierte la lista de listas B en un array de NumPy

    matriz_resultado = matriz_a - matriz_b # resta vectorizada: NumPy resta casilla a casilla
                                           # de forma automatica (operador "-" sobrecargado)

    return matriz_resultado.tolist() # regresa el array a formato lista de listas
```

Explicación línea por línea

Línea	Código	Qué hace / qué significa
1	<code>import numpy as np</code>	Importa la librería NumPy (Numerical Python), especializada en cálculo con vectores y matrices, y le da el alias corto <code>np</code> para escribir menos.
4	<code>def restar_matrices_numpy(A, B):</code>	Misma firma que los métodos anteriores: recibe A y B (listas de listas) y devuelve la resta, para que sea intercambiable con los otros dos métodos.
16	<code>matriz_a = np.array(A)</code>	Convierte la lista de listas A en un <code>ndarray</code> de NumPy: una estructura especial optimizada para operaciones matemáticas masivas.
17	<code>matriz_b = np.array(B)</code>	Igual que la línea anterior, pero para la matriz B.
19	<code>matriz_resultado = matriz_a - matriz_b</code>	Aquí NO hay ningún bucle escrito por nosotros: el operador <code>-</code> está "sobrecargado" por NumPy, así que resta automáticamente cada casilla de <code>matriz_a</code> con la casilla correspondiente de <code>matriz_b</code> , de forma vectorizada e internamente optimizada.
22	<code>return matriz_resultado.tolist()</code>	<code>.tolist()</code> convierte el resultado (que es un <code>ndarray</code> de NumPy) de nuevo a una lista de listas normal de Python, para que el frontend lo pueda mostrar igual que en los otros métodos.

5.3 Frontend — código completo (Tkinter)

Igual que en los métodos anteriores, la interfaz gráfica (menú de tamaño, casillas, botón) es la misma; solo cambia qué función de backend procesa los datos.

```

import tkinter as tk
from tkinter import messagebox
from m3_backend import restar_matrices_numpy # backend del Metodo 3 (usa NumPy)

class VentanaMetodo3:
    """Interfaz grafica (Tkinter) para el Metodo 3: NumPy (vectorizado)."""

    def __init__(self, raiz):
        self.raiz = raiz
        self.raiz.title("Metodo 3 - Resta de Matrices con NumPy")

        self.n = tk.IntVar(value=2)
        self.entradas_a = []
        self.entradas_b = []
        self.n_actual = 2

        self.marco_menu = tk.Frame(raiz, padx=10, pady=10)
        self.marco_menu.pack()
        self.marco_matrices = tk.Frame(raiz, padx=10, pady=10)
        self.marco_matrices.pack()
        self.marco_resultado = tk.Frame(raiz, padx=10, pady=10)
        self.marco_resultado.pack()

        self._crear_menu_tamano()

    def _crear_menu_tamano(self):
        tk.Label(self.marco_menu, text="1) Elige el tamaño de la matriz:",
                 font=("Arial", 11, "bold")).grid(row=0, column=0, columnspan=4, sticky="w")
        tk.Radiobutton(self.marco_menu, text="2 x 2", variable=self.n, value=2).grid(row=1, column=0)
        tk.Radiobutton(self.marco_menu, text="3 x 3", variable=self.n, value=3).grid(row=1, column=1)
        tk.Radiobutton(self.marco_menu, text="N x N ->", variable=self.n, value=-1).grid(row=1, column=2)
        self.entrada_n = tk.Entry(self.marco_menu, width=4)
        self.entrada_n.grid(row=1, column=3)
        tk.Button(self.marco_menu, text="Generar matrices",
                  command=self._dibujar_campos).grid(row=2, column=0, columnspan=4, pady=6)

    def _obtener_n(self):
        if self.n.get() == -1:
            valor = self.entrada_n.get().strip()
            if not valor.isdigit() or int(valor) < 1:
                messagebox.showerror("Error", "Ingresa un entero N valido (N >= 1).")
                return None
            return int(valor)
        return self.n.get()

    def _dibujar_campos(self):
        for widget in self.marco_matrices.winfo_children():
            widget.destroy()
        n = self._obtener_n()
        if n is None:
            return
        self.n_actual = n
        self.entradas_a, self.entradas_b = [], []

        tk.Label(self.marco_matrices, text="Matriz A", font=("Arial", 10, "bold"))
            .grid(row=0, column=0, columnspan=n)
        tk.Label(self.marco_matrices, text="Matriz B", font=("Arial", 10, "bold"))
            .grid(row=0, column=n + 1, columnspan=n)

        for i in range(n):
            fila_a, fila_b = [], []
            for j in range(n):
                celda_a = tk.Entry(self.marco_matrices, width=5, justify="center")
                celda_a.grid(row=i + 1, column=j, padx=2, pady=2)
                celda_a.insert(0, "0")
                fila_a.append(celda_a)

                celda_b = tk.Entry(self.marco_matrices, width=5, justify="center")
                celda_b.grid(row=i + 1, column=n + 1 + j, padx=2, pady=2)
                celda_b.insert(0, "0")
                fila_b.append(celda_b)
            self.entradas_a.append(fila_a)
            self.entradas_b.append(fila_b)

        tk.Button(self.marco_matrices, text="Calcular A - B", bg="#6a1b9a", fg="white",
                  command=self._calcular).grid(row=n + 2, column=0, columnspan=2 * n + 1, pady=10)

```

```

def _calcular(self):
    n = self.n_actual
    try:
        A = [[float(self.entradas_a[i][j].get()) for j in range(n)] for i in range(n)]
        B = [[float(self.entradas_b[i][j].get()) for j in range(n)] for i in range(n)]
    except ValueError:
        messagebox.showerror("Error", "Todos los valores deben ser numeros.")
        return

    resultado = restar_matrices_numpy(A, B) # <-- backend del Metodo 3 (NumPy)
    self._mostrar_resultado(resultado)

def _mostrar_resultado(self, resultado):
    for widget in self.marco_resultado.winfo_children():
        widget.destroy()
    tk.Label(self.marco_resultado, text="Resultado A - B",
             font=("Arial", 10, "bold")).pack()
    texto = "\n".join(" ".join(f"{val:g}" for val in fila) for fila in resultado)
    tk.Label(self.marco_resultado, text=texto, font=("Courier", 12), justify="left").pack()

if __name__ == "__main__":
    raiz = tk.Tk()
    VentanaMetodo3(raiz)
    raiz.mainloop()

```

Diferencia clave respecto a los Frontend anteriores

Línea	Código	Qué hace / qué significa
3	<pre>from m3_backend import restar_matrices_numpy</pre>	Importa la función que usa NumPy.
83	<pre>resultado = restar_matrices_numpy(A, B)</pre>	Se llama al backend del Método 3. Nótese que A y B siguen siendo listas de listas de Python normales: es <i>dentro</i> de la función de backend donde se convierten a arrays de NumPy (ver línea 16-17 del backend).

6. Aplicación integrada final — menú de tamaño y de método

Esta última aplicación reúne **los tres backends** en un solo programa y agrega un segundo menú para elegir, además del tamaño de la matriz (2×2 / 3×3 / $N \times N$), **qué método** (1, 2 o 3) se usará para calcular la resta. Así se puede comparar fácilmente el mismo resultado calculado por las tres formas distintas, sin cambiar de programa.

```
import tkinter as tk
from tkinter import messagebox
import numpy as np

# =====
# BACKEND: los 3 metodos de resta, juntos en un solo lugar
# =====

def restar_bucles(A, B):
    """Metodo 1: bucles for anidados."""
    filas, columnas = len(A), len(A[0])
    resultado = []
    for i in range(filas):
        fila = []
        for j in range(columnas):
            fila.append(A[i][j] - B[i][j])
        resultado.append(fila)
    return resultado

def restar_comprension(A, B):
    """Metodo 2: comprension de listas."""
    filas, columnas = len(A), len(A[0])
    return [[A[i][j] - B[i][j] for j in range(columnas)] for i in range(filas)]

def restar_numpy(A, B):
    """Metodo 3: NumPy (vectorizado)."""
    return (np.array(A) - np.array(B)).tolist()

# Diccionario "menu": asocia el nombre del metodo con su funcion real.
# Esto evita usar una larga cadena de "if/elif/elif" para elegir el metodo.
METODOS = {
    "bucles": restar_bucles,
    "comprension": restar_comprension,
    "numpy": restar_numpy,
}

# =====
# FRONTEND: aplicacion Tkinter con menu de tamano y metodo
# =====

class AplicacionRestaMatrices:
    def __init__(self, raiz):
        self.raiz = raiz
        self.raiz.title("Resta de Matrices - 2x2 / 3x3 / NxN - 3 metodos")

        self.n = tk.IntVar(value=2) # tamano elegido (2, 3 o -1 = "NxN")
        self.metodo = tk.StringVar(value="bucles") # metodo elegido: bucles/comprension/numpy
        self.entradas_a, self.entradas_b = [], []
        self.n_actual = 2

        self.marco_menu = tk.Frame(raiz, padx=10, pady=10)
        self.marco_menu.pack(fill="x")
        self.marco_matrices = tk.Frame(raiz, padx=10, pady=10)
        self.marco_matrices.pack()
        self.marco_resultado = tk.Frame(raiz, padx=10, pady=10)
        self.marco_resultado.pack()

        self._construir_menu()

    # ----- Construye el panel de menu (tamano + metodo) -----
    def _construir_menu(self):
        tk.Label(self.marco_menu, text="Tamano de matriz:",
            font=("Arial", 10, "bold")).grid(row=0, column=0, sticky="w")
        tk.Radiobutton(self.marco_menu, text="2x2", variable=self.n, value=2).grid(row=0, column=1)
        tk.Radiobutton(self.marco_menu, text="3x3", variable=self.n, value=3).grid(row=0, column=2)
        tk.Radiobutton(self.marco_menu, text="NxN ->", variable=self.n, value=-1).grid(row=0, column=3)
        self.entrada_n = tk.Entry(self.marco_menu, width=4)
        self.entrada_n.grid(row=0, column=4)

        tk.Label(self.marco_menu, text="Metodo:",
            font=("Arial", 10, "bold")).grid(row=1, column=0, sticky="w")
        tk.Radiobutton(self.marco_menu, text="1) Bucles", variable=self.metodo,
            value="bucles").grid(row=1, column=1)
        tk.Radiobutton(self.marco_menu, text="2) Comprension", variable=self.metodo,
```

```

        value="compresion").grid(row=1, column=2)
tk.Radiobutton(self.marco_menu, text="3) NumPy", variable=self.metodo,
               value="numpy").grid(row=1, column=3)

tk.Button(self.marco_menu, text="Generar matrices",
         command=self._dibujar_campos).grid(row=2, column=0, columnspan=5, pady=8)

def _obtener_n(self):
    if self.n.get() == -1:
        valor = self.entrada_n.get().strip()
        if not valor.isdigit() or int(valor) < 1:
            messagebox.showerror("Error", "Ingresa un entero N valido (N >= 1).")
            return None
        return int(valor)
    return self.n.get()

# ----- Genera dinamicamente los Entry de A y B segun N -----
def _dibujar_campos(self):
    for widget in self.marco_matrices.wininfo_children():
        widget.destroy()
    n = self._obtener_n()
    if n is None:
        return
    self.n_actual = n
    self.entradas_a, self.entradas_b = [], []

    tk.Label(self.marco_matrices, text="Matriz A", font=("Arial", 10, "bold")
            ).grid(row=0, column=0, columnspan=n)
    tk.Label(self.marco_matrices, text="Matriz B", font=("Arial", 10, "bold")
            ).grid(row=0, column=n + 1, columnspan=n)

    for i in range(n):
        fila_a, fila_b = [], []
        for j in range(n):
            celda_a = tk.Entry(self.marco_matrices, width=5, justify="center")
            celda_a.grid(row=i + 1, column=j, padx=2, pady=2)
            celda_a.insert(0, "0")
            fila_a.append(celda_a)

            celda_b = tk.Entry(self.marco_matrices, width=5, justify="center")
            celda_b.grid(row=i + 1, column=n + 1 + j, padx=2, pady=2)
            celda_b.insert(0, "0")
            fila_b.append(celda_b)
        self.entradas_a.append(fila_a)
        self.entradas_b.append(fila_b)

    tk.Button(self.marco_matrices, text="Calcular A - B", bg="#37474f", fg="white",
            command=self._calcular).grid(row=n + 2, column=0, columnspan=2 * n + 1, pady=10)

# ----- Lee los valores, elige la funcion correcta y calcula -----
def _calcular(self):
    n = self.n_actual
    try:
        A = [[float(self.entradas_a[i][j].get()) for j in range(n)] for i in range(n)]
        B = [[float(self.entradas_b[i][j].get()) for j in range(n)] for i in range(n)]
    except ValueError:
        messagebox.showerror("Error", "Todos los valores deben ser numeros.")
        return

    funcion_resta = METODOS[self.metodo.get()] # elige la funcion segun el radiobutton
    resultado = funcion_resta(A, B)           # ejecuta el backend elegido
    self._mostrar_resultado(resultado)

def _mostrar_resultado(self, resultado):
    for widget in self.marco_resultado.wininfo_children():
        widget.destroy()
    tk.Label(self.marco_resultado, text=f"Resultado A - B (metodo: {self.metodo.get()})",
            font=("Arial", 10, "bold")).pack()
    texto = "\n".join(" ".join(f"{val:g}" for val in fila) for fila in resultado)
    tk.Label(self.marco_resultado, text=texto, font=("Courier", 12), justify="left").pack()

if __name__ == "__main__":
    raiz = tk.Tk()
    AplicacionRestaMatrices(raiz)
    raiz.mainloop()

```

Piezas clave que unen todo

Línea	Código	Qué hace / qué significa
21-24	<pre>METODOS = { "bucles": restar_bucles, "comprension": restar_comprension, "numpy": restar_numpy, }</pre>	Un diccionario que asocia el nombre de cada método (texto) con la función real que lo calcula. Esto evita escribir una larga cadena de <code>if / elif / elif</code> para decidir cuál usar.
58-63	<pre>tk.Radiobutton(..., variable=self.metodo, value="bucles") tk.Radiobutton(..., variable=self.metodo, value="comprension") tk.Radiobutton(..., variable=self.metodo, value="numpy")</pre>	Tres botones de opción comparten la misma variable <code>self.metodo</code> : solo uno puede estar seleccionado a la vez, y su valor (texto) indica qué método usar.
108	<pre>funcion_resta = METODOS[self.metodo.get()]</pre>	<code>self.metodo.get()</code> obtiene el texto del método elegido (por ejemplo "numpy"). Se usa como llave del diccionario <code>METODOS</code> para obtener la función correcta , sin necesidad de comparar manualmente.
109	<pre>resultado = funcion_resta(A, B)</pre>	Se llama a la función obtenida (sea cual sea) exactamente igual, porque las tres funciones de backend reciben los mismos parámetros (A, B) y devuelven el mismo tipo de resultado.

Resumen del flujo en la app integrada:

1. El usuario elige tamaño (2x2 / 3x3 / NxN) y método (Bucles / Comprensión / NumPy).
2. Presiona "Generar matrices" → se dibujan las casillas de A y B según N.
3. Llena los valores y presiona "Calcular A - B".
4. El programa busca en el diccionario `METODOS` la función correcta según el método elegido.
5. Ejecuta esa función con las matrices A y B ingresadas.
6. Muestra el resultado, indicando además con qué método fue calculado.

7. Tabla comparativa de los 3 métodos

Criterio	Método 1: Bucles	Método 2: Comprensión	Método 3: NumPy
Líneas de código	Varias (más largo)	Compacto (1 expresión)	Muy compacto
Facilidad para un principiante	Muy alta: se ve cada paso	Media: requiere entender la sintaxis	Alta una vez que se conoce NumPy
Dependencias externas	Ninguna (Python puro)	Ninguna (Python puro)	Requiere instalar <code>numpy</code>
Rendimiento con matrices grandes	Más lento	Similar a bucles, algo más rápido	Mucho más rápido (vectorizado)
Qué tan visible es el recorrido	Totalmente explícito	Implícito, en una línea	Oculto (lo resuelve NumPy internamente)

Criterio	Método 1: Bucles	Método 2: Comprensión	Método 3: NumPy
Uso recomendado	Aprender la lógica desde cero	Código Python "idiomático" y limpio	Cálculo numérico serio / matrices grandes

8. Glosario de términos de Python usados

def	Palabra clave que define (crea) una función.
return	Entrega el resultado de una función a quien la llamó.
for x in range(n)	Bucle que repite un bloque de código, asignando a <code>x</code> los valores 0, 1, 2, ... hasta <code>n-1</code> .
len(lista)	Función que devuelve cuántos elementos tiene una lista.
lista.append(x)	Agrega el valor <code>x</code> al final de la lista.
lista[i][j]	Accede al elemento de la fila <code>i</code> y columna <code>j</code> cuando la lista contiene otras listas (matriz).
Comprensión de listas	Sintaxis <code>[expresión for elemento in secuencia]</code> que construye una lista nueva en una sola línea.
class	Define un "molde" (tipo de objeto) que agrupa datos (atributos) y funciones (métodos) relacionados.
self	Dentro de una clase, hace referencia al objeto actual; permite guardar y leer datos propios de cada ventana/instancia.
tk.Frame	Widget "contenedor" invisible de Tkinter que agrupa otros widgets.
tk.Entry	Widget de Tkinter: una casilla de texto donde el usuario puede escribir un valor.
tk.Label	Widget de Tkinter para mostrar texto (no editable) en pantalla.
tk.Button	Widget de botón; el parámetro <code>command</code> indica qué función se ejecuta al hacer clic.
tk.Radiobutton	Botón de opción; varios comparten una misma variable y solo uno puede estar activo.
IntVar / StringVar	"Cajas" especiales de Tkinter que guardan un número o texto y se sincronizan automáticamente con los widgets.
.grid(row=, column=)	Ubica un widget dentro de su contenedor, en una cuadrícula de filas y columnas.
widget.destroy()	Elimina un widget de la pantalla (se usa para "limpiar" antes de volver a dibujar).
try / except ValueError	Intenta ejecutar un bloque de código y, si ocurre un error de conversión de tipo (por ejemplo texto que no es número), ejecuta el bloque alternativo en vez de que el programa se caiga.
np.array(lista)	(NumPy) Convierte una lista de listas de Python en una matriz optimizada de NumPy.
.tolist()	(NumPy) Convierte una matriz de NumPy de regreso a una lista de listas de Python normal.
Vectorización	Técnica de realizar una operación sobre todos los elementos de un arreglo "de una sola vez", usando código optimizado internamente, en vez de un bucle explícito en Python.

9. Conclusiones

Los tres métodos presentados resuelven exactamente el mismo problema —la resta de dos matrices, casilla por casilla— pero representan tres niveles distintos de abstracción en Python:

- El **Método 1** es el más didáctico: permite ver, paso a paso, cómo se recorre una matriz con índices explícitos. Es el punto de partida ideal para entender la lógica.
- El **Método 2** muestra cómo Python permite expresar la misma lógica de forma más corta y "elegante", una vez que la lógica de los bucles ya se domina.
- El **Método 3** introduce el concepto de vectorización con NumPy, fundamental cuando se trabaja con matrices grandes o cálculo numérico intensivo.

La aplicación integrada final (sección 6) demuestra además un principio importante de diseño de software: **separar el backend (la lógica de cálculo) del frontend (la interfaz gráfica)**. Gracias a esa separación, fue posible intercambiar los tres métodos de cálculo sin tener que modificar la interfaz gráfica en absoluto.